

Nesterov Smoothing for Structured Non-Smooth Convex Optimization

Michał Szewczak, Sebastian Trojan

Contents

1	Theoretical Background	2
1.1	Problem class	2
1.2	Smoothing the inner maximization	2
1.3	Gradient of the smoothed objective	3
1.4	Accelerated minimization	3
1.5	Dual averaging and stopping criterion	4
1.6	Complexity bound	4
2	Problem Classes	5
2.1	Matrix games	5
2.2	Continuous location problem	6
2.3	maximum of absolute values problem	7
2.4	Sum of absolute values problem	8
3	Changing the Smoothing Parameter	8
4	Experiments and Results	9
4.1	Reproduction of Nesterov’s matrix game experiment	9
4.2	Experiments for the other examples	11
4.2.1	Continuous location problem	11
4.2.2	maximum of absolute values problem	12
4.2.3	Sum of absolute values problem	13
4.3	Overall comparison of the reproduction-style experiments	15
4.4	Fixed-size epsilon-scaling experiment	15
4.4.1	Iterations versus $1/\varepsilon$	16
4.4.2	Log-log plot and fitted exponent	17
4.5	Experiment with changing the smoothing parameter μ	18
4.5.1	Iteration comparison	19
4.5.2	Iteration speedup	20
4.5.3	Gap history	21
5	Discussion	24
6	Conclusion	24

Abstract

This report describes an implementation of Nesterov's smoothing technique for structured non-smooth convex optimization problems. The method replaces a non-smooth objective with a smooth approximation obtained by regularizing an auxiliary maximization problem, and then applies an accelerated first-order method. The implementation is tested on matrix games, the continuous location problem, and two piecewise-linear objectives: the maximum of absolute values and the sum of absolute values. The experiments study primal-dual gap convergence, the empirical dependence of the iteration count on the required accuracy, and a practical continuation strategy for changing the smoothing parameter.

1 Theoretical Background

1.1 Problem class

Nesterov's smoothing technique [1] applies to a class of convex non-smooth optimization problems of the form

$$f^* = \min_{x \in Q_1} f(x),$$

where Q_1 is a closed convex feasible set and f is convex but may be non-smooth.

The key assumption is that the non-smooth part of the objective can be written as a maximization over an auxiliary variable:

$$f(x) = \hat{f}(x) + \max_{u \in Q_2} \left\{ \langle Ax, u \rangle - \hat{\phi}(u) \right\}.$$

Here Q_2 is another closed convex set, A is a linear operator, $\hat{f}$ is a smooth convex function, and $\hat{\phi}$ is convex. The non-smoothness comes from the inner maximization over u .

The variable x is the primal variable. The variable u is an auxiliary or dual variable. For a fixed x , the inner problem finds the worst or most active dual response.

1.2 Smoothing the inner maximization

The central idea is to replace the hard maximum by a smooth approximation. Choose a prox-function $d_2(u)$ on Q_2 , strongly convex with parameter $\sigma_2 > 0$, and define

$$D_2 = \max_{u \in Q_2} d_2(u).$$

For a smoothing parameter $\mu > 0$, define

$$f_\mu(x) = \max_{u \in Q_2} \left\{ \langle Ax, u \rangle - \hat{\phi}(u) - \mu d_2(u) \right\}.$$

The extra term $-\mu d_2(u)$ penalizes extreme choices of u . This makes the inner maximization better behaved and makes the resulting function differentiable.

The smoothed approximation satisfies the uniform error bound

$$f_\mu(x) \leq f(x) \leq f_\mu(x) + \mu D_2.$$

Therefore,

$$0 \leq f(x) - f_\mu(x) \leq \mu D_2.$$

Thus smaller μ gives a more accurate approximation of the original objective. However, smaller μ also makes the smoothed objective harder to optimize.

1.3 Gradient of the smoothed objective

For each x , define the smoothed inner maximizer

$$u_\mu(x) = \arg \max_{u \in Q_2} \left\{ \langle Ax, u \rangle - \hat{\phi}(u) - \mu d_2(u) \right\}.$$

Nesterov shows that the smoothed function is differentiable and

$$\nabla f_\mu(x) = A^* u_\mu(x).$$

In matrix notation this is usually $A^\top u_\mu(x)$.

If the smooth part $\hat{f}$ is present, then

$$\nabla \bar{f}_\mu(x) = \nabla \hat{f}(x) + A^* u_\mu(x).$$

The Lipschitz constant of this gradient is bounded by

$$L_\mu = M + \frac{\|A\|_{1,2}^2}{\mu \sigma_2},$$

where M is the Lipschitz constant of $\nabla \hat{f}$. In the examples implemented in this project, $\hat{f} = 0$, so $M = 0$.

This formula shows the basic trade-off:

$$\mu \downarrow \Rightarrow \text{better approximation, but larger } L_\mu.$$

1.4 Accelerated minimization

After smoothing, the method minimizes the smooth problem

$$\min_{x \in Q_1} f_\mu(x).$$

Let $d_1(x)$ be a prox-function on Q_1 , strongly convex with parameter $\sigma_1 > 0$, and define

$$D_1 = \max_{x \in Q_1} d_1(x).$$

The accelerated method maintains three sequences:

$$x_k, \quad y_k, \quad z_k.$$

At iteration k , compute

$$g_k = \nabla f_\mu(x_k).$$

The local point is

$$y_k = T_{Q_1}(x_k),$$

where

$$T_{Q_1}(x_k) = \arg \min_{y \in Q_1} \left\{ \langle g_k, y - x_k \rangle + \frac{L}{2} \|y - x_k\|^2 \right\}.$$

In Euclidean geometry this becomes the projected gradient step

$$y_k = \text{Proj}_{Q_1} \left(x_k - \frac{1}{L} g_k \right).$$

The method also accumulates previous gradients. With

$$\alpha_k = \frac{k+1}{2},$$

define

$$G_k = \sum_{i=0}^k \alpha_i g_i.$$

The aggregate point is

$$z_k = \arg \min_{x \in Q_1} \left\{ \frac{L}{\sigma_1} d_1(x) + \langle G_k, x \rangle \right\}.$$

The point y_k is based on the current local gradient, while z_k summarizes all past gradient information.

The next search point is obtained by mixing these two points:

$$x_{k+1} = \frac{2}{k+3} z_k + \frac{k+1}{k+3} y_k.$$

This is the acceleration step.

1.5 Dual averaging and stopping criterion

At each iteration the method computes a smoothed dual response

$$u_k = u_\mu(x_k).$$

The final dual candidate is a weighted average:

$$\hat{u} = \frac{\sum_{k=0}^N \alpha_k u_k}{\sum_{k=0}^N \alpha_k}.$$

Since Q_2 is convex, $\hat{u} \in Q_2$.

The dual value is denoted by $\phi(\hat{u})$. Weak duality gives

$$\phi(\hat{u}) \leq f^* \leq f(\hat{x}),$$

where $\hat{x}$ is the final primal point. Therefore

$$f(\hat{x}) - \phi(\hat{u})$$

is a primal-dual gap. The algorithm stops when

$$f(\hat{x}) - \phi(\hat{u}) \leq \varepsilon.$$

This gives a certificate that $\hat{x}$ is ε -optimal.

1.6 Complexity bound

For the considered class of problems, Nesterov proves a bound of the form

$$0 \leq f(\hat{x}) - \phi(\hat{u}) \leq \frac{4\|A\|_{1,2}}{N+1} \sqrt{\frac{D_1 D_2}{\sigma_1 \sigma_2}} + \frac{4MD_1}{\sigma_1(N+1)^2}.$$

When $\hat{f} = 0$, we have $M = 0$, and the bound simplifies to

$$f(\hat{x}) - \phi(\hat{u}) \leq \frac{4\|A\|_{1,2}}{N+1} \sqrt{\frac{D_1 D_2}{\sigma_1 \sigma_2}}.$$

Thus, to obtain accuracy ε , it is sufficient to take

$$N+1 \geq \frac{4\|A\|_{1,2}}{\varepsilon} \sqrt{\frac{D_1 D_2}{\sigma_1 \sigma_2}}.$$

Therefore, the theoretical iteration complexity is

$$N = O\left(\frac{1}{\varepsilon}\right).$$

This improves over the classical subgradient rate $O(1/\varepsilon^2)$ for this structured class of non-smooth problems.

2 Problem Classes

The examples considered below all fit the same general smoothing framework, but they use different feasible sets and prox-functions. The first two examples are matrix games and continuous location. The last two examples are both piecewise-linear objectives built from absolute residuals. They are reported separately because the maximum of absolute values uses an entropy smoothing over a simplex, while the sum of absolute values uses a separable Huber-type smoothing.

2.1 Matrix games

The matrix game problem is

$$\min_{x \in \Delta_n} \max_{u \in \Delta_m} u^\top A x,$$

where $A \in \mathbb{R}^{m \times n}$, and Δ_n, Δ_m are probability simplexes.

For fixed x ,

$$\max_{u \in \Delta_m} u^\top A x = \max_j (A x)_j.$$

Hence the problem is equivalent to

$$\min_{x \in \Delta_n} \max_j (A x)_j.$$

The entropy prox-functions are

$$d_1(x) = \ln n + \sum_{i=1}^n x_i \ln x_i,$$

and

$$d_2(u) = \ln m + \sum_{j=1}^m u_j \ln u_j.$$

Thus

$$D_1 = \ln n, \quad D_2 = \ln m.$$

The smoothed objective is

$$f_\mu(x) = \mu \ln \left(\frac{1}{m} \sum_{j=1}^m \exp \left(\frac{(A x)_j}{\mu} \right) \right).$$

The smoothed dual response is

$$u_j(x) = \frac{\exp((A x)_j / \mu)}{\sum_{\ell=1}^m \exp((A x)_\ell / \mu)}.$$

The gradient is

$$\nabla f_\mu(x) = A^\top u(x).$$

For a feasible dual strategy u , the dual value is

$$\phi(u) = \min_i (A^\top u)_i.$$

The primal-dual gap is

$$\max_j (A x)_j - \min_i (A^\top u)_i.$$

2.2 Continuous location problem

The continuous location problem is

$$\min_{\|x\|_2 \leq r} \sum_{j=1}^p w_j \|x - c_j\|_2.$$

Here x is the facility location, c_j are city locations, and $w_j > 0$ are weights.

Each norm can be written as

$$\|x - c_j\|_2 = \max_{\|u_j\|_2 \leq 1} \langle x - c_j, u_j \rangle.$$

Thus the dual variable is a collection

$$U = (u_1, \dots, u_p), \quad \|u_j\|_2 \leq 1.$$

The primal prox-function is

$$d_1(x) = \frac{1}{2} \|x\|_2^2,$$

so

$$D_1 = \frac{1}{2} r^2.$$

Let

$$P = \sum_{j=1}^p w_j.$$

Then

$$D_2 = \frac{1}{2} P.$$

The smoothed distance is

$$\psi_\mu(t) = \begin{cases} \frac{t^2}{2\mu}, & 0 \leq t \leq \mu, \\ t - \frac{\mu}{2}, & t > \mu. \end{cases}$$

Therefore

$$f_\mu(x) = \sum_{j=1}^p w_j \psi_\mu(\|x - c_j\|_2).$$

The smoothed dual direction is

$$u_j(x) = \begin{cases} \frac{x - c_j}{\mu}, & \|x - c_j\|_2 \leq \mu, \\ \frac{x - c_j}{\|x - c_j\|_2}, & \|x - c_j\|_2 > \mu. \end{cases}$$

The gradient is

$$\nabla f_\mu(x) = \sum_{j=1}^p w_j u_j(x).$$

For a feasible U , the dual value is

$$\phi(U) = -r \left\| \sum_{j=1}^p w_j u_j \right\|_2 - \sum_{j=1}^p w_j \langle c_j, u_j \rangle.$$

The primal-dual gap is

$$\sum_{j=1}^p w_j \|x - c_j\|_2 - \phi(U).$$

2.3 maximum of absolute values problem

The maximum of absolute values problem is

$$\min_{\|x\|_2 \leq r} \max_{1 \leq j \leq m} |a_j^\top x - b_j|.$$

It minimizes the worst absolute residual.

Each absolute value can be rewritten as

$$|a_j^\top x - b_j| = \max\{a_j^\top x - b_j, -a_j^\top x + b_j\}.$$

Define

$$\tilde{A} = \begin{bmatrix} A \\ -A \end{bmatrix}, \quad \tilde{b} = \begin{bmatrix} -b \\ b \end{bmatrix}.$$

Then

$$\max_j |a_j^\top x - b_j| = \max_{\ell=1, \dots, 2m} \left\{ \tilde{a}_\ell^\top x + \tilde{b}_\ell \right\}.$$

Equivalently,

$$f(x) = \max_{u \in \Delta_{2m}} u^\top (\tilde{A}x + \tilde{b}).$$

The primal prox-function is

$$d_1(x) = \frac{1}{2} \|x\|_2^2, \quad D_1 = \frac{1}{2} r^2.$$

The dual prox-function is entropy on Δ_{2m} :

$$d_2(u) = \ln(2m) + \sum_{\ell=1}^{2m} u_\ell \ln u_\ell, \quad D_2 = \ln(2m).$$

The smoothed objective is

$$f_\mu(x) = \mu \ln \left(\frac{1}{2m} \sum_{\ell=1}^{2m} \exp \left(\frac{\tilde{a}_\ell^\top x + \tilde{b}_\ell}{\mu} \right) \right).$$

Equivalently,

$$f_\mu(x) = \mu \ln \left(\frac{1}{m} \sum_{j=1}^m \xi \left(\frac{a_j^\top x - b_j}{\mu} \right) \right),$$

where

$$\xi(t) = \frac{1}{2} (e^t + e^{-t}).$$

The dual value is

$$\phi(u) = \tilde{b}^\top u - r \|\tilde{A}^\top u\|_2.$$

The primal-dual gap is

$$\max_j |a_j^\top x - b_j| - \phi(u).$$

2.4 Sum of absolute values problem

The fourth problem is

$$\min_{\|x\|_2 \leq r} \sum_{j=1}^m |a_j^\top x - b_j|.$$

This is related to least absolute deviations and robust regression.

Each absolute value has the representation

$$|a_j^\top x - b_j| = \max_{|u_j| \leq 1} u_j(a_j^\top x - b_j).$$

Therefore

$$\sum_{j=1}^m |a_j^\top x - b_j| = \max_{\|u\|_\infty \leq 1} \sum_{j=1}^m u_j(a_j^\top x - b_j).$$

The primal feasible set is the Euclidean ball, with

$$d_1(x) = \frac{1}{2} \|x\|_2^2, \quad D_1 = \frac{1}{2} r^2.$$

The smoothed absolute value is

$$\psi_\mu(t) = \begin{cases} \frac{t^2}{2\mu}, & |t| \leq \mu, \\ |t| - \frac{\mu}{2}, & |t| > \mu. \end{cases}$$

Thus

$$f_\mu(x) = \sum_{j=1}^m \psi_\mu(a_j^\top x - b_j).$$

The smoothed dual coordinate is

$$u_j(x) = \begin{cases} \frac{a_j^\top x - b_j}{\mu}, & |a_j^\top x - b_j| \leq \mu, \\ \text{sign}(a_j^\top x - b_j), & |a_j^\top x - b_j| > \mu. \end{cases}$$

The gradient is

$$\nabla f_\mu(x) = A^\top u(x).$$

For a feasible dual vector u , the dual value is

$$\phi(u) = -b^\top u - r \|A^\top u\|_2.$$

The primal-dual gap is

$$\sum_{j=1}^m |a_j^\top x - b_j| - \phi(u).$$

3 Changing the Smoothing Parameter

The theoretical method uses a fixed smoothing parameter. For a target accuracy ε , the typical choice is

$$\mu = \frac{\varepsilon}{2D_2}.$$

This gives

$$\mu D_2 = \frac{\varepsilon}{2},$$

so half of the error budget is assigned to smoothing error.

In practice, the value of μ can also be changed during the optimization. The motivation is that large μ gives an easier and smoother problem, while small μ gives a more accurate approximation. A continuation strategy starts with

$$\mu_0 = c\mu_{\text{final}},$$

where $c > 1$, and then decreases μ geometrically:

$$\mu_{t+1} = \rho\mu_t, \quad 0 < \rho < 1.$$

In the implementation, a typical choice is

$$\mu_0 = 8\mu_{\text{final}}, \quad \rho = 0.5.$$

This gives the sequence

$$8\mu_{\text{final}}, \quad 4\mu_{\text{final}}, \quad 2\mu_{\text{final}}, \quad \mu_{\text{final}}.$$

At intermediate stages, the method stops when the gap is comparable to the current smoothing error:

$$f(\hat{x}) - \phi(\hat{u}) \leq c_{\text{stage}}\mu_t D_2.$$

At the final stage, the full condition is required:

$$f(\hat{x}) - \phi(\hat{u}) \leq \varepsilon.$$

The fixed- μ method is the version directly covered by the theory. Changing μ is a practical heuristic intended to reduce the computational cost by solving a sequence of easier smoothed problems.

4 Experiments and Results

This section presents the computational experiments. The experiments have four purposes. First, the matrix game experiment from Nesterov's paper is reproduced and used as a reference. Second, analogous reproduction-style experiments are performed for the continuous location problem and the two piecewise-linear absolute-value objectives. Third, a fixed-size epsilon-scaling experiment is used to check the dependence of the iteration count on the accuracy parameter ε . Finally, the fixed smoothing parameter is compared with a continuation strategy in which μ is decreased during optimization.

In all experiments, the stopping criterion is the primal-dual gap condition

$$f(\hat{x}) - \phi(\hat{u}) \leq \varepsilon.$$

Therefore, convergence means that the obtained solution is certified to be ε -optimal.

4.1 Reproduction of Nesterov's matrix game experiment

The first experiment reproduces the matrix game experiment from Nesterov's paper. The problem is

$$\min_{x \in \Delta_n} \max_{u \in \Delta_m} u^\top A x,$$

where $A \in \mathbb{R}^{m \times n}$. Random matrices were generated with entries sampled uniformly from $[-1, 1]$.

The tested grid was

ε	m	n
10^{-2}	100, 300, 1000	100, 300, 1000, 3000, 10000
10^{-3}	100, 300, 1000	100, 300, 1000, 3000, 10000
10^{-4}	100, 300, 1000	100, 300, 1000, 3000

Similarly to the original paper, the results are presented as separate tables for each value of ε . Rows correspond to m , columns correspond to n , and each cell contains

iterations / percentage of predicted iterations.

$m \backslash n$	100	300	1000	3000	10000
100	765 / 42%	934 / 46%	1 108 / 49%	1 187 / 49%	1 281 / 49%
300	690 / 34%	910 / 40%	1 126 / 45%	1 268 / 47%	1 475 / 51%
1000	761 / 34%	873 / 35%	1 086 / 39%	1 284 / 43%	1 457 / 46%

Table 1: Matrix game reproduction results for $\varepsilon = 0.01$. Each cell shows iterations / percentage of predicted iterations.

$m \backslash n$	100	300	1000	3000	10000
100	7 123 / 39%	8 426 / 41%	8 928 / 40%	10 191 / 42%	10 080 / 39%
300	7 337 / 36%	8 639 / 38%	10 154 / 40%	11 302 / 42%	12 022 / 41%
1000	7 526 / 33%	8 828 / 35%	10 157 / 37%	11 595 / 39%	14 167 / 44%

Table 2: Matrix game reproduction results for $\varepsilon = 0.001$. Each cell shows iterations / percentage of predicted iterations.

$m \backslash n$	100	300	1000	3000
100	68 505 / 37%	64 512 / 31%	77 136 / 34%	74 024 / 30%
300	60 932 / 30%	78 680 / 34%	83 844 / 33%	92 862 / 34%
1000	70 778 / 31%	77 644 / 31%	93 346 / 34%	102 925 / 35%

Table 3: Matrix game reproduction results for $\varepsilon = 0.0001$. Each cell shows iterations / percentage of predicted iterations.

The aggregate results are summarized in Table 4.

ε	Instances	Avg. iter.	Max iter.	Avg. % pred.	Max gap	Avg. time
10^{-4}	12	78 766	102 925	33.00%	0.000100	465s
10^{-3}	15	9 765	14 167	39.07%	0.001000	140s
10^{-2}	15	1 080	1 475	43.13%	0.009997	6.83s

Table 4: Aggregate summary of the matrix game reproduction experiment.

Conclusion. The reproduced matrix game experiment confirms the main practical behavior reported in the paper. All tested instances reached the required primal-dual gap. The number of iterations was significantly smaller than the theoretical prediction. In the generated results,

the method used only a fraction of the predicted number of iterations, which shows that the theoretical estimate is conservative in practice.

The results also show the expected dependence on ε . When ε is decreased by one order of magnitude, the number of iterations increases approximately by one order of magnitude. This is consistent with the theoretical complexity $O(1/\varepsilon)$. The theoretical bound is therefore conservative, but it captures the correct scaling.

4.2 Experiments for the other examples

After reproducing the matrix game experiment, analogous experiments were performed for the other implemented examples. The goal was to check whether the same smoothing framework behaves similarly for different non-smooth structures.

The tested problems were:

1. continuous location problem,
2. maximum of absolute values problem,
3. sum of absolute values problem.

In each case, the same type of information was recorded: number of iterations, predicted number of iterations, percentage of the predicted bound used, runtime, and final primal-dual gap.

4.2.1 Continuous location problem

The continuous location problem is

$$\min_{\|x\|_2 \leq r} \sum_{j=1}^p w_j \|x - c_j\|_2.$$

City locations were generated randomly inside an Euclidean ball and positive weights were assigned to the cities. The radius was fixed to $r = 1$.

The tested grid was

ε	number of cities	dimension	r
10^{-2}	10, 50	2, 5	1
10^{-3}	10, 50	2, 5	1
10^{-4}	10, 50	2, 5	1

The results are presented separately for each value of ε . Rows correspond to the number of cities p , columns correspond to the dimension d , and each cell contains

iterations / percentage of predicted iterations.

$p \backslash d$	2	5
10	566 / 35%	932 / 47%
50	3 606 / 35%	2 847 / 29%

Table 5: Continuous location results for $\varepsilon = 0.01$. Each cell shows iterations / percentage of predicted iterations.

$p \backslash d$	2	5
10	10 890 / 62%	11 407 / 49%
50	31 653 / 33%	28 151 / 30%

Table 6: Continuous location results for $\varepsilon = 0.001$. Each cell shows iterations / percentage of predicted iterations.

$p \backslash d$	2	5
10	48 121 / 28%	75 747 / 36%
50	320 867 / 34%	241 660 / 25%

Table 7: Continuous location results for $\varepsilon = 0.0001$. Each cell shows iterations / percentage of predicted iterations.

The aggregate results are shown in Table 8.

ε	Instances	Avg. iter.	Max iter.	Avg. % pred.	Max gap	Avg. time
10^{-4}	4	171 599	320 867	30.64%	0.000100	31.8s
10^{-3}	4	20 525	31 653	43.74%	0.001000	3.86s
10^{-2}	4	1 988	3 606	36.70%	0.009997	0.35s

Table 8: Summary of continuous location experiments.

Conclusion. All continuous location instances converged. The achieved gaps were below the required tolerances. The method required substantially fewer iterations than the theoretical upper bound predicted, which confirms that the bound is conservative for this example as well.

The iteration count increased significantly as ε decreased, which is consistent with the theoretical dependence on $1/\varepsilon$. Runtime was moderate because the tested continuous location instances were relatively small.

4.2.2 maximum of absolute values problem

The maximum of absolute values problem is

$$\min_{\|x\|_2 \leq r} \max_{1 \leq j \leq m} |a_j^\top x - b_j|.$$

Random matrices A and vectors b were generated with entries sampled uniformly from $[-1, 1]$. The radius was fixed to $r = 1$.

The tested grid was

ε	m	n	r
10^{-2}	100, 300	50, 200	1
10^{-3}	100, 300	50, 200	1
10^{-4}	100, 300	50, 200	1

The results are presented separately for each value of ε . Rows correspond to m , columns correspond to n , and each cell contains

iterations / percentage of predicted iterations.

$m \backslash n$	50	200
100	1 793 / 60%	3 018 / 53%
300	1 773 / 51%	3 788 / 60%

Table 9: maximum of absolute values results for $\varepsilon = 0.01$. Each cell shows iterations / percentage of predicted iterations.

$m \backslash n$	50	200
100	16 673 / 54%	13 649 / 24%
300	16 135 / 48%	33 671 / 53%

Table 10: maximum of absolute values results for $\varepsilon = 0.001$. Each cell shows iterations / percentage of predicted iterations.

$m \backslash n$	50	200
100	152 449 / 49%	116 635 / 20%
300	167 001 / 50%	249 861 / 40%

Table 11: maximum of absolute values results for $\varepsilon = 0.0001$. Each cell shows iterations / percentage of predicted iterations.

The aggregate results are shown in Table 12.

ε	Instances	Avg. iter.	Max iter.	Avg. % pred.	Max gap	Avg. time
10^{-4}	4	171 486	249 861	39.81%	0.000100	10.6s
10^{-3}	4	20 032	33 671	44.76%	0.001000	1.26s
10^{-2}	4	2 593	3 788	56.06%	0.009998	0.16s

Table 12: Summary of maximum of absolute values experiments.

Conclusion. All maximum of absolute values instances converged. The final gaps were below the requested tolerances. The results show that entropy smoothing of the maximum over affine functions works effectively in practice.

Compared with the theoretical prediction, the actual number of iterations was again smaller. This confirms the same pattern observed for matrix games and continuous location: the theoretical bound is useful for predicting the scaling, but it overestimates the absolute number of iterations needed on these random instances.

4.2.3 Sum of absolute values problem

The sum of absolute values problem is

$$\min_{\|x\|_2 \leq r} \sum_{j=1}^m |a_j^\top x - b_j|.$$

Random matrices A and vectors b were generated with entries sampled uniformly from $[-1, 1]$. The radius was fixed to $r = 1$.

The tested grid was

ε	m	n	r
10^{-2}	100, 300	50, 200	1
10^{-3}	100, 300	50, 200	1
10^{-4}	100, 300	50, 200	1

The results are presented separately for each value of ε . Rows correspond to m , columns correspond to n , and each cell contains

iterations / percentage of predicted iterations.

$m \backslash n$	50	200
100	5 146 / 27%	5 561 / 21%
300	27 750 / 58%	9 896 / 16%

Table 13: Sum of absolute values results for $\varepsilon = 0.01$. Each cell shows iterations / percentage of predicted iterations.

$m \backslash n$	50	200
100	40 986 / 22%	61 784 / 23%
300	284 262 / 60%	156 544 / 25%

Table 14: Sum of absolute values results for $\varepsilon = 0.001$. Each cell shows iterations / percentage of predicted iterations.

$m \backslash n$	50	200
100	677 811 / 34%	140 591 / 5%
300	2 711 368 / 58%	1 263 846 / 20%

Table 15: Sum of absolute values results for $\varepsilon = 0.0001$. Each cell shows iterations / percentage of predicted iterations.

The aggregate results are shown in Table 16.

ε	Instances	Avg. iter.	Max iter.	Avg. % pred.	Max gap	Avg. time
10^{-4}	4	1 198 404	2 711 368	29.45%	0.000100	54.9s
10^{-3}	4	135 894	284 262	32.52%	0.001000	6.43s
10^{-2}	4	12 088	27 750	30.43%	0.010000	0.51s

Table 16: Summary of sum of absolute values experiments.

Conclusion. All sum of absolute values instances converged. However, this problem required substantially more iterations than the other examples.

This is expected because the objective sums all absolute residuals, while the max-absolute problem only considers the largest residual. The sum structure can lead to larger Lipschitz constants for the smoothed objective. Nevertheless, the actual iteration count was still significantly below the theoretical prediction.

4.3 Overall comparison of the reproduction-style experiments

The results across all four examples are summarized in Table 17.

Problem	Instances	Avg. iter.	Avg. % pred.	Max gap	Avg. time
Matrix game	42	26 378	38.78%	0.009997	185s
Continuous location	12	64 704	37.03%	0.009997	12.0s
Maximum of absolute values	12	64 704	46.88%	0.009998	4.01s
Sum of absolute values	12	448 795	30.80%	0.010000	20.6s

Table 17: Overall comparison of reproduction-style experiments.

Conclusion. The same smoothing framework works for all tested examples. In every case, the method reached the requested primal-dual gap. The theoretical iteration bound was conservative for all examples: the method typically required much less than the predicted number of iterations.

The experiments also show that the practical difficulty depends strongly on the structure of the problem. The sum of absolute values problem required the most iterations, while the maximum of absolute values and continuous location problems were cheaper for the tested sizes. Matrix games became expensive mainly because the largest tested matrices had very large dimensions.

4.4 Fixed-size epsilon-scaling experiment

The previous experiments used grids with several problem dimensions. To examine the dependence on ε more directly, an additional fixed-size experiment was performed.

For each problem class and each random seed, one random instance was generated and reused for all values of ε . Thus, only the accuracy parameter changed. This isolates the dependence of the iteration count on ε .

The tested values were

$$\varepsilon \in \{10^{-1}, 5 \cdot 10^{-2}, 10^{-2}, 5 \cdot 10^{-3}, 10^{-3}, 5 \cdot 10^{-4}, 10^{-4}\}.$$

The fixed problem sizes were:

Problem	Size
Matrix game	$m = 300, n = 1000$
Continuous location	$p = 50, d = 5, r = 1$
Maximum of absolute values	$m = 300, n = 200, r = 1$
Sum of absolute values	$m = 300, n = 200, r = 1$

The numerical results are summarized in Table 18.

Problem	ε	$1/\varepsilon$	Mean iter.	Std iter.	Mean gap	Max gap
Matrix game	10^{-1}	10	79	8.44	0.099210	0.099693
Matrix game	$5 \cdot 10^{-2}$	20	213	5.70	0.049820	0.049973
Matrix game	10^{-2}	100	1 097	12.52	0.009987	0.009996
Matrix game	$5 \cdot 10^{-3}$	200	2 141	35.85	0.004997	0.005000
Matrix game	10^{-3}	1 000	9 806	212.18	0.001000	0.001000
Matrix game	$5 \cdot 10^{-4}$	2 000	18 634	513.54	0.000500	0.000500
Matrix game	10^{-4}	10 000	82 064	2 622	0.000100	0.000100
Continuous location	10^{-1}	10	337	18.26	0.099485	0.099937
Continuous location	$5 \cdot 10^{-2}$	20	662	37.22	0.049880	0.049966
Continuous location	10^{-2}	100	3 338	300.01	0.009996	0.009999
Continuous location	$5 \cdot 10^{-3}$	200	6 633	609.15	0.004998	0.004999
Continuous location	10^{-3}	1 000	33 049	3 085	0.001000	0.001000
Continuous location	$5 \cdot 10^{-4}$	2 000	66 469	5 902	0.000500	0.000500
Continuous location	10^{-4}	10 000	332 524	29 668	0.000100	0.000100
Maximum of absolute values	10^{-1}	10	445	5.93	0.099316	0.099969
Maximum of absolute values	$5 \cdot 10^{-2}$	20	841	5.26	0.049825	0.049918
Maximum of absolute values	10^{-2}	100	3 830	22.48	0.009998	0.010000
Maximum of absolute values	$5 \cdot 10^{-3}$	200	7 453	24.63	0.004999	0.005000
Maximum of absolute values	10^{-3}	1 000	34 107	357.33	0.001000	0.001000
Maximum of absolute values	$5 \cdot 10^{-4}$	2 000	63 575	1 016	0.000500	0.000500
Maximum of absolute values	10^{-4}	10 000	240 017	9 058	0.000100	0.000100
Sum of absolute values	10^{-1}	10	1 562	89.13	0.099690	0.099962
Sum of absolute values	$5 \cdot 10^{-2}$	20	2 813	286.48	0.049985	0.049999
Sum of absolute values	10^{-2}	100	12 791	1 782	0.010000	0.010000
Sum of absolute values	$5 \cdot 10^{-3}$	200	25 048	3 830	0.005000	0.005000
Sum of absolute values	10^{-3}	1 000	122 206	19 730	0.001000	0.001000
Sum of absolute values	$5 \cdot 10^{-4}$	2 000	242 824	40 122	0.000500	0.000500
Sum of absolute values	10^{-4}	10 000	1 207 464	201 485	0.000100	0.000100

Table 18: Summary of the fixed-size epsilon-scaling experiment.

4.4.1 Iterations versus $1/\varepsilon$

The first plot shows the mean number of iterations as a function of $1/\varepsilon$.

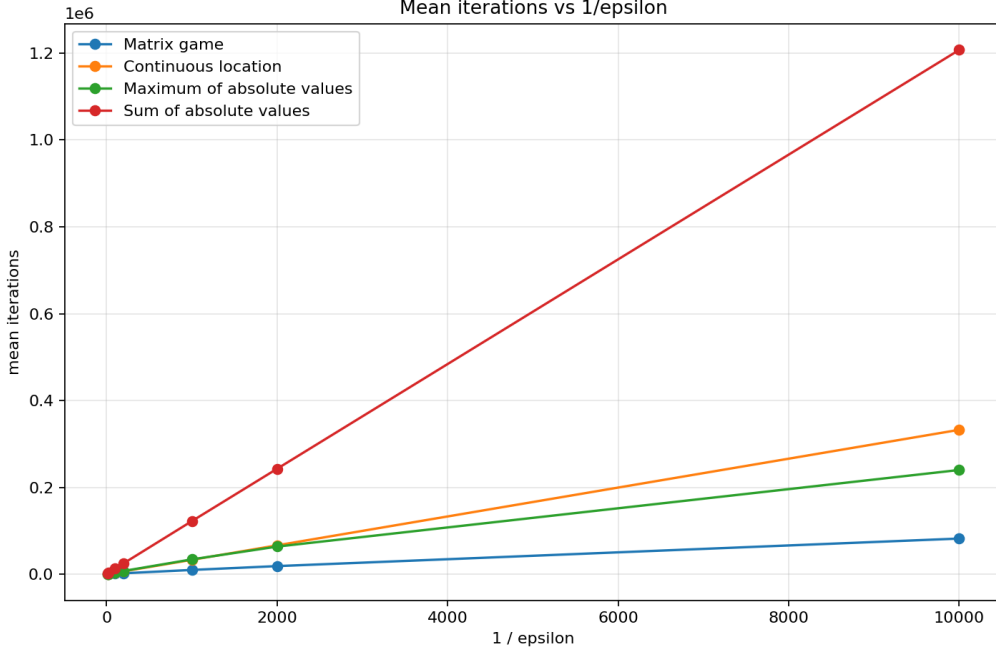


Figure 1: Mean number of iterations versus $1/\varepsilon$.

The goal of this plot is to check the theoretical prediction visually. If

$$N(\varepsilon) \approx C \frac{1}{\varepsilon},$$

then the curves should be approximately linear. The plot shows this behavior for all four implemented examples. The sum of absolute values problem has the largest constant factor, while the matrix game has the smallest one in this experiment. However, the main point is that the curves grow approximately linearly with $1/\varepsilon$, which supports the expected complexity

$$N = O(1/\varepsilon).$$

4.4.2 Log-log plot and fitted exponent

To examine the empirical rate more precisely, the iteration counts were also analyzed on a log-log scale. If the number of iterations satisfies a relation of the form

$$N(\varepsilon) \approx C \varepsilon^{-p},$$

then taking logarithms gives

$$\log N = \log C + p \log(1/\varepsilon).$$

Thus, the exponent p can be estimated as the slope of a linear regression between $\log N$ and $\log(1/\varepsilon)$. The theoretical complexity $N = O(1/\varepsilon)$ corresponds to

$$p = 1.$$

The log-log plot is shown in Figure 2. The approximately linear shape of the curves confirms that a power-law model is a reasonable description of the observed dependence on ε .

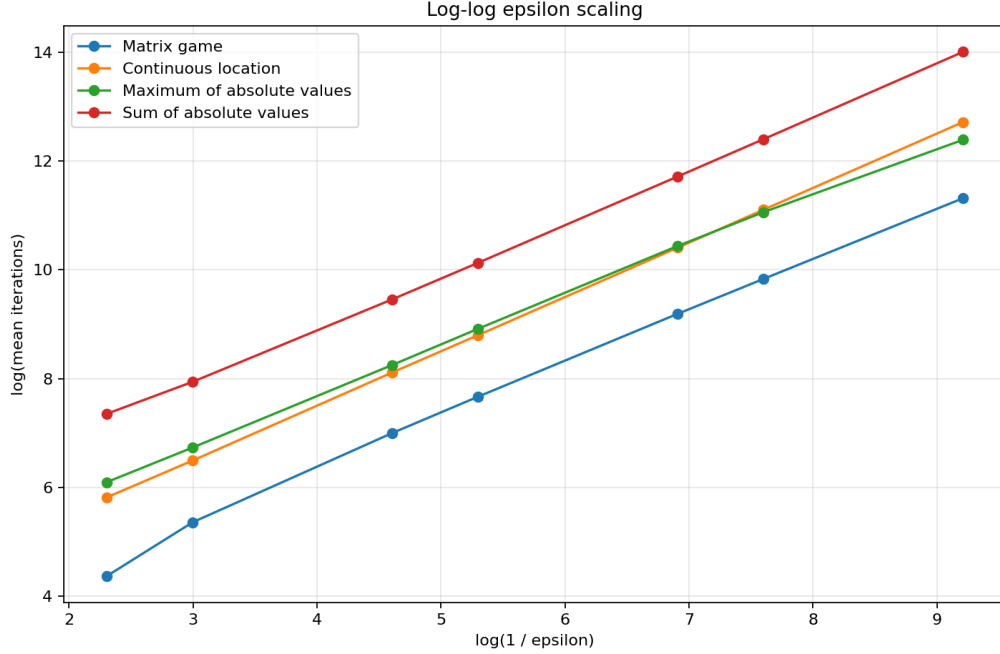


Figure 2: Log-log plot for the epsilon-scaling experiment.

The fitted exponents are shown in Table 19. The table reports the estimated exponent $\hat{p}$ and its standard error.

Problem	$\hat{p}$	SE
Matrix game	0.9897	0.0096
Continuous location	0.9986	0.0054
Maximum of absolute values	0.9208	0.0042
Sum of absolute values	0.9649	0.0096

Table 19: Fitted exponents in the empirical model $N(\varepsilon) \approx C\varepsilon^{-p}$.

The fitted exponents for the matrix game and continuous location problem are very close to 1, which is consistent with the theoretical $O(1/\varepsilon)$ complexity. For the maximum of absolute values problem and the sum of absolute values problem, the fitted exponents are slightly below 1. This means that, for these random instances, the observed iteration counts grew slightly slower than the worst-case $1/\varepsilon$ rate.

Overall, the log-log analysis supports the theoretical prediction. The observed rates are close to $1/\varepsilon$, and in some cases slightly better. This is consistent with the interpretation that the theoretical complexity bound is a worst-case estimate and may be conservative on randomly generated instances.

4.5 Experiment with changing the smoothing parameter μ

The previous experiments used the fixed smoothing parameter prescribed by the theory. In this experiment, this fixed- μ strategy is compared with a continuation strategy, where the smoothing parameter is changed during the optimization process.

The motivation is the trade-off between accuracy and smoothness. A smaller μ gives a better approximation of the original non-smooth problem, but it also increases the Lipschitz constant of the smoothed objective. Therefore, a smaller μ makes the problem harder to optimize.

The continuation strategy tries to exploit this by starting with a larger and easier smoothing parameter, and then gradually decreasing it.

Two strategies were compared: the fixed- μ strategy and the continuation strategy.

In the fixed strategy, the algorithm uses the final theoretical value μ_{final} from the beginning. In the continuation strategy, the initial value is

$$\mu_0 = 8\mu_{\text{final}},$$

and then μ is decreased geometrically:

$$\mu_{t+1} = 0.5\mu_t.$$

Thus, the continuation sequence is

$$8\mu_{\text{final}}, \quad 4\mu_{\text{final}}, \quad 2\mu_{\text{final}}, \quad \mu_{\text{final}}.$$

At intermediate stages, the algorithm stops when the gap is comparable to the current smoothing error. At the final stage, the usual stopping condition is used:

$$f(\hat{x}) - \phi(\hat{u}) \leq \varepsilon.$$

The experiment was performed for all four problem classes, using the same fixed problem sizes as in the epsilon-scaling experiment:

Problem	Size
Matrix game	$m = 300, n = 1000$
Continuous location	$p = 50, d = 5, r = 1$
Maximum of absolute values	$m = 300, n = 200, r = 1$
Sum of absolute values	$m = 300, n = 200, r = 1$

The tested accuracies were

$$\varepsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}.$$

For each problem, each value of ε , and each random seed, the same random instance was solved twice: once with fixed μ , and once with the continuation strategy. This makes the comparison fair, since both methods solve the same instance.

4.5.1 Iteration comparison

Table 20 reports the average number of iterations for both strategies.

Problem	ε	μ mode	Mean iter.	Std iter.
Continuous location	10^{-4}	Continuation	379 813	37 174
Continuous location	10^{-4}	Fixed μ	306 630	29 498
Continuous location	10^{-3}	Continuation	37 330	10 378
Continuous location	10^{-3}	Fixed μ	30 237	8 283
Continuous location	10^{-2}	Continuation	2 923	1 247
Continuous location	10^{-2}	Fixed μ	3 240	483.64
Matrix game	10^{-4}	Continuation	115 059	2 313
Matrix game	10^{-4}	Fixed μ	88 760	1 864
Matrix game	10^{-3}	Continuation	12 853	416.61
Matrix game	10^{-3}	Fixed μ	9 997	326.88
Matrix game	10^{-2}	Continuation	1 466	29.78
Matrix game	10^{-2}	Fixed μ	1 126	30.57
Maximum of absolute values	10^{-4}	Continuation	331 092	26 995
Maximum of absolute values	10^{-4}	Fixed μ	260 380	18 927
Maximum of absolute values	10^{-3}	Continuation	44 448	536.44
Maximum of absolute values	10^{-3}	Fixed μ	34 493	385.07
Maximum of absolute values	10^{-2}	Continuation	5 059	68.72
Maximum of absolute values	10^{-2}	Fixed μ	3 790	81.63
Sum of absolute values	10^{-4}	Continuation	1 679 708	193 437
Sum of absolute values	10^{-4}	Fixed μ	1 297 689	147 250
Sum of absolute values	10^{-3}	Continuation	150 866	7 829
Sum of absolute values	10^{-3}	Fixed μ	115 690	6 887
Sum of absolute values	10^{-2}	Continuation	18 853	2 015
Sum of absolute values	10^{-2}	Fixed μ	14 448	1 536

Table 20: Comparison of fixed μ and continuation μ .

The results show that continuation did not improve the iteration count for most tested cases. For the matrix game, maximum of absolute values problem, and sum of absolute values problem, the continuation method required more iterations than the fixed- μ method for all tested accuracies. The same behavior is also visible for the continuous location problem for $\varepsilon = 10^{-3}$ and $\varepsilon = 10^{-4}$. The only case where continuation improved the average iteration count was continuous location with $\varepsilon = 10^{-2}$.

This indicates that, for the tested instances and the chosen continuation parameters, the extra stages introduced by changing μ did not compensate for the additional iterations needed across stages.

4.5.2 Iteration speedup

To make the comparison clearer, Table 21 reports the iteration speedup

$$\text{speedup} = \frac{\text{iterations for fixed } \mu}{\text{iterations for continuation } \mu}.$$

Thus, values greater than 1 mean that continuation used fewer iterations, while values below 1 mean that fixed μ was better.

Problem	ε	Mean iter. speedup	Min iter. speedup	Max iter. speedup
Continuous location	10^{-4}	0.807	0.806	0.810
Continuous location	10^{-3}	0.811	0.806	0.818
Continuous location	10^{-2}	1.28	0.804	1.89
Matrix game	10^{-4}	0.771	0.768	0.774
Matrix game	10^{-3}	0.778	0.772	0.782
Matrix game	10^{-2}	0.768	0.759	0.778
Maximum of absolute values	10^{-4}	0.787	0.775	0.792
Maximum of absolute values	10^{-3}	0.776	0.770	0.781
Maximum of absolute values	10^{-2}	0.749	0.740	0.759
Sum of absolute values	10^{-4}	0.773	0.764	0.779
Sum of absolute values	10^{-3}	0.767	0.756	0.778
Sum of absolute values	10^{-2}	0.766	0.758	0.773

Table 21: Iteration speedup of continuation relative to fixed μ .

The speedup table confirms the previous observation. Most speedup values are below 1, usually around 0.75–0.81. This means that continuation typically required about 20%–30% more iterations than the fixed strategy. The only average speedup above 1 occurs for the continuous location problem with $\varepsilon = 10^{-2}$, where the mean speedup is approximately 1.28. However, even in this case the minimum speedup is below 1, so the improvement was not consistent across all seeds.

Overall, this experiment suggests that the fixed- μ method is more efficient for the tested settings.

4.5.3 Gap history

In addition to final iteration counts, the primal-dual gap was tracked during optimization for representative runs with

$$\varepsilon = 10^{-4}, \quad \text{seed} = 0.$$

This gives more detailed information about the behavior of both methods. The following plots show the primal-dual gap as a function of the iteration number. The y -axis is logarithmic, and the horizontal dotted line marks the target accuracy $\varepsilon = 10^{-4}$. For the continuation method, vertical dotted lines indicate changes of the smoothing parameter μ .

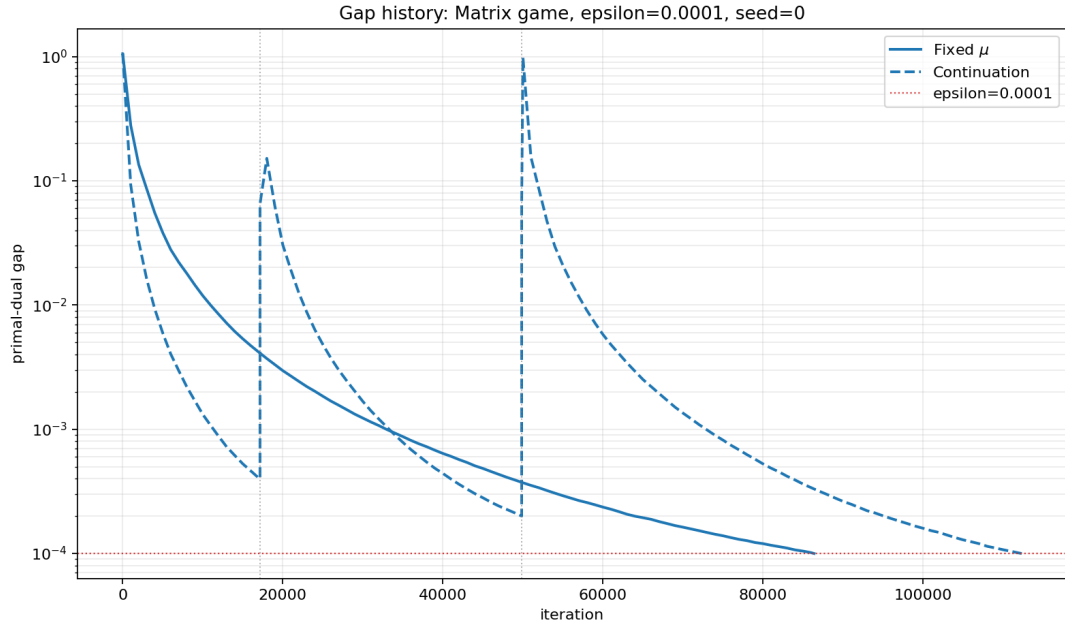


Figure 3: Gap history for the matrix game with $\varepsilon = 10^{-4}$.

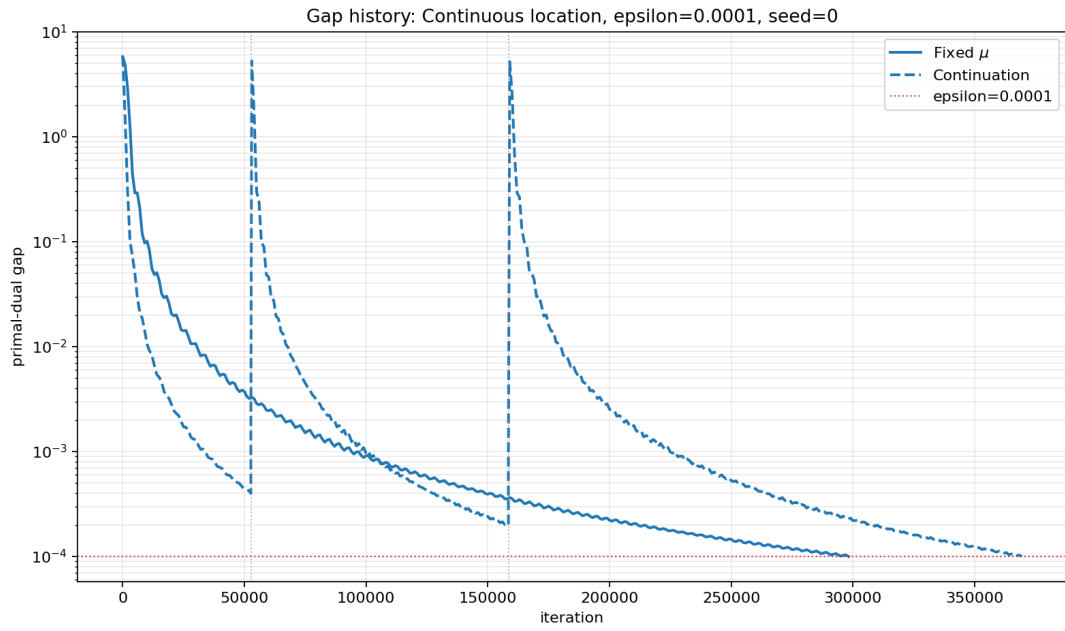


Figure 4: Gap history for the continuous location problem with $\varepsilon = 10^{-4}$.

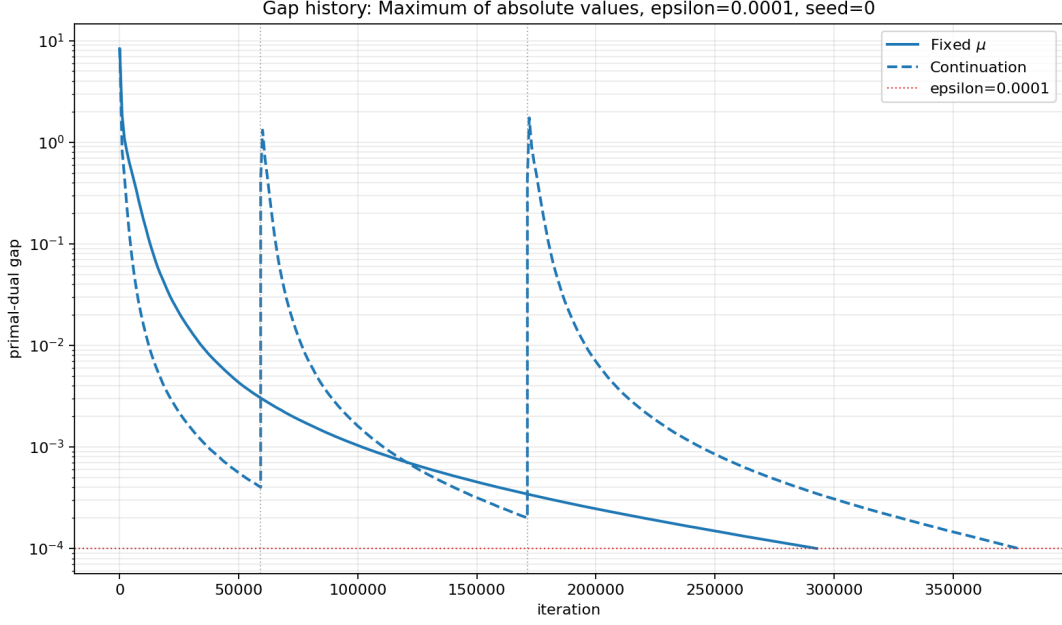


Figure 5: Gap history for the maximum of absolute values problem with $\varepsilon = 10^{-4}$.

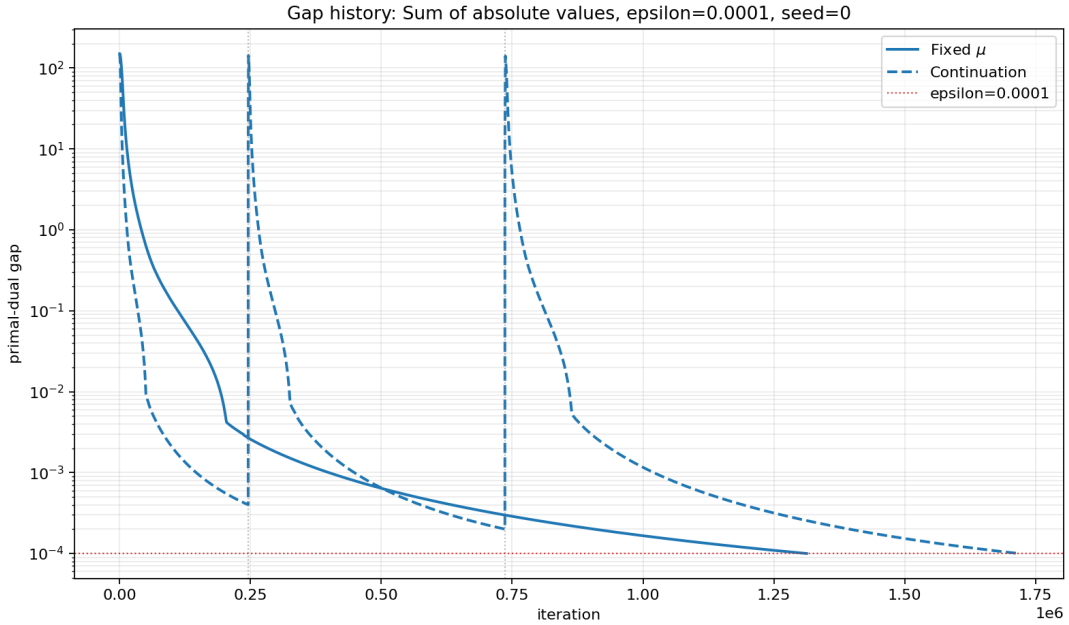


Figure 6: Gap history for the sum of absolute values problem with $\varepsilon = 10^{-4}$.

The plots show that continuation reduces the gap quickly inside each stage, but after each decrease of μ , the gap increases again. This is expected: changing μ changes the smoothed objective and makes the approximation closer to the original non-smooth function. Therefore, the current point may no longer have as small a gap for the new, less smoothed problem.

For the tested instances, the fixed- μ method reaches the target line earlier than continuation. This is especially clear for the matrix game, maximum of absolute values problem, and sum of absolute values problem. In the continuous location problem, the difference is smaller, but the fixed method still reaches the final tolerance earlier for the displayed $\varepsilon = 10^{-4}$ run.

Conclusion. The changing- μ experiment shows that continuation is not automatically better than using the fixed theoretical smoothing parameter. Although larger values of μ make the early stages easier, the method must still solve several stages and the gap can increase when μ is decreased. In the tested configuration, with start factor 8 and decay 0.5, this additional work usually outweighed the benefit of starting with a smoother problem.

Therefore, for the implemented examples and tested parameters, the fixed- μ strategy is preferable. Continuation remains a potentially useful heuristic, but its performance depends on the choice of the starting factor, decay rate, and stage stopping rule. Further experiments with different continuation parameters would be needed before concluding that changing μ is beneficial.

5 Discussion

The experiments confirm the main practical message of Nesterov’s smoothing method. The primal-dual gap was below the requested tolerance in all tested reproduction-style experiments, so the stopping criterion provided a meaningful certificate of solution quality. At the same time, the actual number of iterations was consistently below the theoretical upper bound. This is expected, because the bound is a worst-case estimate, while the experiments use randomly generated instances.

The epsilon-scaling experiment gives the clearest evidence for the theoretical rate. For the matrix game and continuous location problem, the fitted exponents are very close to 1, which agrees with the predicted $O(1/\varepsilon)$ complexity. For the maximum of absolute values and sum of absolute values problems, the fitted exponents are below 1. This does not contradict the theory; instead, it means that the observed growth was slightly better than the worst-case rate on the tested instances.

The results also show that the constant hidden in the complexity estimate is important. The sum of absolute values problem required many more iterations than the other examples, even though its dependence on ε remained close to linear. This is consistent with the structure of the objective: unlike the maximum of absolute values, which depends only on the largest residual, the sum of absolute values aggregates all residuals and can therefore lead to a larger effective Lipschitz constant after smoothing.

The continuation experiment shows that changing μ is not automatically beneficial. Although larger values of μ make the early smoothed problems easier, the method must solve several stages. Moreover, when μ is reduced, the gap can increase because the smoothed approximation changes. With the tested parameters $\mu_0 = 8\mu_{\text{final}}$ and decay factor 0.5, the fixed strategy was usually better in terms of iteration count. Continuation should therefore be treated as a heuristic whose performance depends on the stage schedule and stopping rule.

6 Conclusion

This project implemented Nesterov’s smoothing technique for several structured non-smooth convex optimization problems. The method was applied to matrix games, the continuous location problem, and two piecewise-linear absolute-value objectives. In each case, the non-smooth objective was represented through an auxiliary maximization problem, smoothed by adding a prox-term, and minimized using an accelerated first-order method.

The numerical experiments support the theoretical behavior of the method. All tested instances reached the required primal-dual gap, and the observed iteration counts were substantially smaller than the theoretical worst-case predictions. The fixed-size epsilon-scaling experiment showed an approximately linear dependence on $1/\varepsilon$, in agreement with the theoretical $O(1/\varepsilon)$ complexity. The experiments also demonstrated that the same smoothing framework

can be applied to different problem structures, although the practical number of iterations depends strongly on the particular objective.

The tested continuation strategy for changing the smoothing parameter did not improve performance for most instances. The fixed theoretical smoothing parameter was usually more efficient. This suggests that, for the implemented examples, the basic fixed- μ method is the preferable default choice. Continuation remains an interesting practical modification, but further tuning of the initial smoothing parameter, decay factor, and stage stopping criterion would be needed to make it competitive.

Overall, the implementation and experiments confirm the usefulness of Nesterov's smoothing framework: it gives a systematic way to solve structured non-smooth convex problems with a certified primal-dual gap and with empirical iteration growth consistent with the theoretical $O(1/\varepsilon)$ rate.

References

- [1] Yurii Nesterov. Smooth minimization of non-smooth functions. *Mathematical Programming*, 103(1):127–152, 2005.